

Dense Random Forest for Subgroup Analysis

This vignette demonstrates how to use the dense random forest method for subgroup analysis. The Amgen Study 309 is used here as an illustrative example.

Set training parameters

Splitting rule for dense random forests.

Fit cox model in each splitting,

$$h(t|M, W) = h_0(t) \times \exp\{\gamma_1 V + \gamma_2 W + \gamma_3 V \times W\}$$

them, the splitting rule is:

$$G(s) = \omega_1 \times (a_1 - 0.5)/2 + (1 - \omega_1) \times (a_2/\omega_2)$$

where ω_1 is the weight parameter, ω_2 is the den parameter. a_1 is the C-Index, a_2 is the Z score of γ_3 in the Cox model.

mtry: Number of candidate variables randomly selected at each split,

ntree: Number of trees to grow,

nsplit: Number of random split points per variable,

nodesize: Minimum terminal node size,

nodedepth: Maximum tree depth,

den_util: den parameter, please use 3.5 as default value,

weight_util: weight parameter.

For more information, see the help documentation.

```
mtry_list = c(2, 3)
ntree = 500
nsplit = c(20, 30, 40)
nodesize = c(50, 70, 100)
nodedepth = c(2, 3)
den_util = 3.5
weight_util = c(30, 40, 50)
```

Then, combine these parameters as the combinatons,

```
para_list = base::expand.grid(mtry_list, ntree, nodesize, nodedepth, nsplit, den_util, weight_util)
colnames(para_list) = c("mtry", "ntree", "nodesize", "nodedepth", "nsplit", "den_util", "weight_util")
df = para_list
```

Here we train the random forest with the above parameters. The following function can use parallel technique.

```

f_simulation <- function(i){
  options(rf.cores=1, mc.cores = 1)

  study_id = "309"

  ### the path is used to save membership matrix
  result_save_path = ""

  if(!dir.exists(result_save_path)){ dir.create(result_save_path)}

  result_folder_path = paste0(result_save_path, "/", study_id)
  if(!dir.exists(result_folder_path)){ dir.create(result_folder_path)}

  ### load data
  data(crc_adsl, package = "randomForestSRC")
  dat <- base::subset(crc_adsl, crc_adsl$StudyID==study_id)
  dat <- dat[!is.na(dat$B_ECOG),]
  dat <- dat[!is.na(dat$KRAS),]
  dat <- dat[!is.na(dat$AGE),]
  dat$TRT <- as.factor(dat$TRT)
  data <- dat[, c("SUBJID", "OS", "OSevent", "KRAS", "AGE", "TRT", "B_ECOG", "ARM", "PFS", "PFSevent")] #
  train.data = data[, c("SUBJID", "OS", "OSevent", "KRAS", "AGE", "B_ECOG", "TRT", "PFS", "PFSevent", "ARM")]

  train.data$TRT = ifelse(train.data$TRT == "FOLFOX alone", yes = 0, no = 1)
  train.data$TRT = as.factor(train.data$TRT)
  train.data$KRAS <- as.factor(train.data$KRAS)
  train.data$B_ECOG <- as.factor(train.data$B_ECOG)
  train.data = filter(train.data, B_ECOG == "1" | B_ECOG == "0")
  train.data$B_ECOG <- as.factor(train.data$B_ECOG)
  x_name = c("KRAS", "B_ECOG", "AGE")

  df = para_list

  data.tmp = train.data

  # train model
  para.mtry = df[i, "mtry"]
  para.ntree = df[i, "ntree"]
  para.nodesize = df[i, "nodesize"]
  para.nodedepth = df[i, "nodedepth"]
  para.nsplits = df[i, "nsplits"]
  para.den_util = df[i, "den_util"]
  para.weight_util = df[i, "weight_util"]
  para.seed = 1
  if(para.nodedepth == 0)
  {
    para.nodedepth = NULL
  }

  ## train random forests with different parameter set up
  rf.object = tryCatch(expr = {rfsrc(Surv(OS, OSevent)~KRAS+B_ECOG+AGE,
                                     data = data.tmp, ntree = para.ntree,
                                     nodesize = para.nodesize,
                                     nodedepth = para.nodedepth,

```

```

mtry=para.mtry,
nsplit = para.nsplit, splitrule = "custom5",
block.size = NULL, statistics = TRUE,
forest = TRUE, membership = TRUE,
weight_util = (para.weight_util/100),
den_util = para.den_util,
treatment = data.tmp$TRT,
xvar.wt = xvar_weight, var.used = "all.trees"}},
error = function(e){0})

if (is.list(rf.object)){
  arg <- list(na.action="na.impute", prox.dist.type="all")
  rf.object.predict <- predict.rfsrc(object=rf.object,
                                   newdata=data.tmp,
                                   na.action = arg$na.action,
                                   membership=TRUE, proximity = FALSE,
                                   treatment_flag = TRUE)

  predict_membership = rf.object.predict$membership
  membership_save_path = paste0(result_folder_path, "/",
                                "para_id_", i, "_mtry_", para.mtry,
                                "_ntree_", para.ntree, "_nodesize_",
                                para.nodesize, "_nsplit_",
                                para.nsplit, "para_weight_util_",
                                para.weight_util, "_nodedepth_",
                                para.nodedepth, ".csv")

  rio::export(predict_membership, membership_save_path)
  base::rm(rf.object)
  base::rm(rf.object.predict)
  base::rm(predict_membership)
  base::rm(membership_save_path)
}
}

```

calculate membership by random forest with custom5 splitting rule

It can be calculated in parallel.

```

for (i in 1:dim(df)[1]){
  f_simulation(i)
}

```

combine memberships.

This code is used to combine membership matrix from the above the step. The inputs are a series of membership matrix, the output is the ensembled membership. (The outputs are too big, so the example only show codes.)

```

folder_path = ""

membership = NULL

## combine all membership matrix
for (i in 1:dim(df)[1]){

```

```

para.mtry = df[i, "mtry"]
para.ntree = df[i, "ntree"]
para.nodesize = df[i, "nodesize"]
para.nodedepth = df[i, "nodedepth"]
para.nsplits = df[i, "nsplits"]
para.den_util = df[i, "den_util"]
para.weight_util = df[i, "weight_util"]
para.seed = 1
for (j in 1:3){
  membership_save_path = paste0(folder_path, "/",
                                "para_id_", i, "_mtry_", para.mtry,
                                "_ntree_", para.ntree, "_nodesize_",
                                para.nodesize, "_nsplits_",
                                para.nsplits, "para_weight_util_",
                                para.weight_util, "_nodedepth_",
                                para.nodedepth, ".csv")

  membership_tmp = rio::import(membership_save_path)
  membership = rbind.data.frame(membership, membership_tmp)
}
}

```

compute proximity from membership

```
proximity = rfsrc_compute_proximity(membership = membership, inbag=NULL, oob='ALL')
```

Here, we load the pre-computed proximity.

```

data(proximity, package = "randomForestSRC")

study_id = "309"
data(crc_adsl, package = "randomForestSRC")
dat <- base::subset(crc_adsl, crc_adsl$StudyID==study_id)
dat <- dat[!is.na(dat$B_ECOG),]
dat <- dat[!is.na(dat$KRAS),]
dat <- dat[!is.na(dat$AGE),]
dat$TRT <- as.factor(dat$TRT)
data <- dat[, c("SUBJID", "OS", "OSevent", "KRAS", "AGE", "TRT", "B_ECOG", "ARM", "PFS", "PFSevent")] # T
train.data = data[, c("SUBJID", "OS", "OSevent", "KRAS", "AGE", "B_ECOG", "TRT", "PFS", "PFSevent", "ARM")

train.data$TRT = ifelse(train.data$TRT == "FOLFOX alone", yes = 0, no = 1)
train.data$TRT = as.factor(train.data$TRT)
train.data$KRAS <- as.factor(train.data$KRAS)
train.data$B_ECOG <- as.factor(train.data$B_ECOG)
train.data = filter(train.data, B_ECOG == "1" | B_ECOG == "0")
train.data$B_ECOG <- as.factor(train.data$B_ECOG)
x_name = c("KRAS", "B_ECOG", "AGE")
dat = train.data

## transform proximity to distance matrix
distance = 1 - proximity
tsne_result = Rtsne::Rtsne(distance, dims=2, is_distance=TRUE, verbose=FALSE,
                           max_iter = 5000, theta = 0)

```

```

## K-Means Clustering
kmeans_result_2 = kmeans(tsne_result$Y, centers = 2, iter.max = 50, nstart = 30)
kmeans_result_3 = kmeans(tsne_result$Y, centers = 3, iter.max = 50, nstart = 30)
kmeans_result_4 = kmeans(tsne_result$Y, centers = 4, iter.max = 50, nstart = 30)
kmeans_result_5 = kmeans(tsne_result$Y, centers = 5, iter.max = 50, nstart = 30)

dat$kmeans2 = as.factor(kmeans_result_2$cluster)
dat$kmeans3 = as.factor(kmeans_result_3$cluster)
dat$kmeans4 = as.factor(kmeans_result_4$cluster)
dat$kmeans5 = as.factor(kmeans_result_5$cluster)

pval_flag = 2
result = list()

for (j in 2:5){
  x_name = c("B_ECOG", "KRAS", "AGE")
  c_name = paste0("kmeans", j)
  profiles <- tree_fit(Y=dat[,c_name], X=dat[,x_name], seed=1234, maxdepth = 2)
  if(is.null(profiles)){
    next
  }
  for (k in 1:length(profiles$trees)){
    dat = predict_path(profiles$trees[[k]], newdata = dat)

    data_tmp = dat
    ## calculate p leaf
    fit_surv3 = coxph(Surv(OS, OSevent)~TRT, data = data_tmp)
    fit_surv2 = coxph(Surv(OS, OSevent)~leaf*TRT, data = data_tmp)
    pval = as.numeric(na.omit(stats::anova(fit_surv2, fit_surv3)[[4]]))

    if(pval < pval_flag){
      pval_flag = pval
      result[[1]] = profiles
      result[[2]] = pval_flag
      result[[3]] = j
      result[[4]] = k
    }
  }
}
}

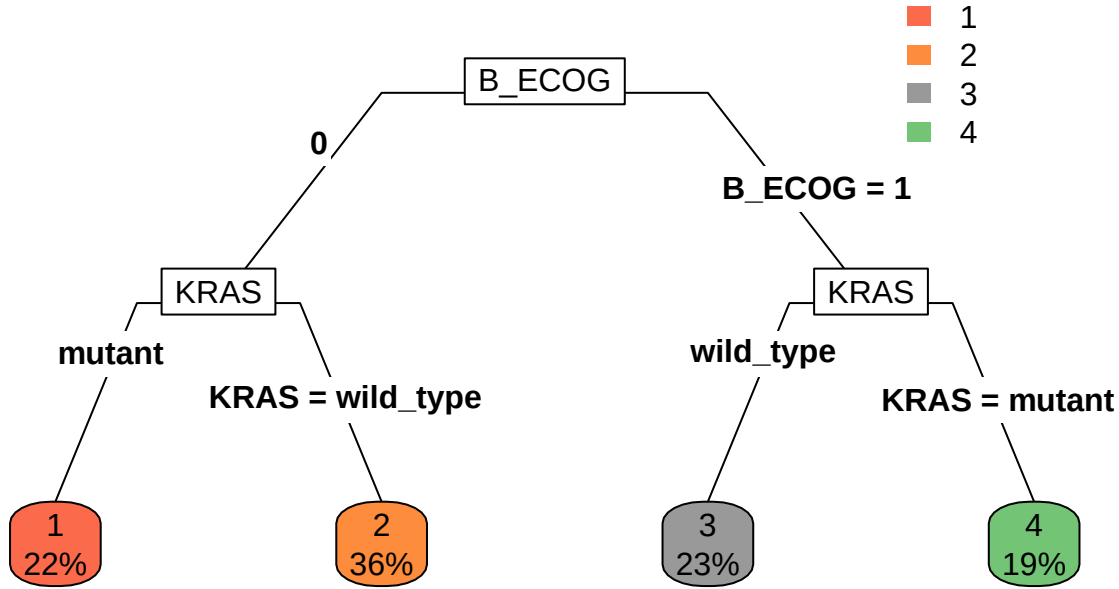
```

Visualize profile identified by dense random forests

Each terminal node represents a discovered subgroup; the split variables and percentages indicate subgroup profiles.

```
plot_profile(result[[1]]$trees[[result[[4]]]])
```

Subtype Profile



We use minimum p_{leaf} to select best decision tree. The detailed definition can be found at <https://arxiv.org/abs/2505.17917>.

```
cat(result[[2]])
#> 7.849685e-13
```

Calibration

We simulate NULL scenario by permute current data, then records the p_{leaf} , the following results are pre-computing results.

```
data("p_leaf_list", package = "randomForestSRC")
```

By comparing the data with the NULL scenario's p_{leaf} , we can exclude non-heterogeneous cases. Here, we control the type I error rate at 1.

```
quantile(p_leaf_list,0.01)
#> 1%
#> 0.0004303215
```

In study 309, the p_{leaf} is smaller than calibration threshold, so there is heterogeneity. The subtype discovery result see the decision tree.